

Universidade do Minho

Escola de Engenharia

Mestrado em Engenharia de
Telecomunicações e Informática
Escola de Engenharia

Relatório do TP1

Serviços de Rede e Aplicações Multimédia [METI]

Algoritmo LZW-HX

Grupo 3 constituído por:

Luís Pedro Cerqueira Baptista, PG61434



Bernardo Gualdino Salgado, PG59972



Data de Entrega: 18 de Abril de 2026

2025/2026

Índice

1. Introdução e Arquitetura do Software.....	4
1.1 Objetivos e Abordagem	4
1.2 Arquitetura do Código	4
1.3 Gestão de Memória e Processamento por Blocos	5
2. Estratégias Escolhidas e Mecanismos Adotados	6
2.1 Versão Base: Pesquisa Linear	6
2.2 Versão Otimizada: Otimização com Hash Tables	6
2.3 Versão Final: Árvores de Prefixos (Tries)	6
3. Explicação das funções implementadas	7
3.1 update_dictionary: A Política de Substituição e de Recurso	8
3.2 find_longest_match: Ambiguidade e Resolução de Sub-Padrões	8
4. Resultados dos testes efetuados	9
4.1 Cenário de Validação Teórica (Alfabeto Reduzido: K=2).....	9
4.2 Cenário Base (Dicionário: 65536 entradas)	10
4.3 Cenário com Dicionário Reduzido (4096 entradas).....	10
4.4 Cenário de Adaptação (4096 entradas com Reset)	11
5. Conclusões e Melhorias	13
.6. Referências Bibliográficas	14
7. Declaração de Uso de IA e Responsabilidade.....	14

Índice de Figuras

Figura 1 - Diagrama de blocos	5
Figura 2 - Validação da extração dos prefixos Pa e Pb no cenário de alfabeto reduzido (Modo Debug).....	9

Índice de Tabelas

Tabela 1 - Comparação do desempenho das estruturas de pesquisa num cenário de validação teórica (Alfabeto reduzido, K=2).....	10
Tabela 2 - Métricas de desempenho e compressão do algoritmo LZW-HX face ao LZW Clássico (Dicionário: 65536 entradas).....	10
Tabela 3 - Impacto da política de reinicialização (reset) no tempo de execução e rendimento do dicionário (Limite: 4096 entradas).....	11
Tabela 4 - Resultados da simulação com o ficheiro zip.....	12
Tabela 5 - Resultados da simulação com o ficheiro JPEG	12
Tabela 6 - Resultados da simulação com o ficheiro pdf	12

1. Introdução e Arquitetura do Software

1.1 Objetivos e Abordagem

O objetivo principal do Trabalho Prático N°1 - Compressão LZW-HX v0.8[1] consistiu na implementação do algoritmo LZW-HX, caracterizado pela geração de novos padrões através da junção de sequências conhecidas (Pa|Pb) e por uma política de substituição dinâmica de prefixos, para quando o limite do dicionário é atingido.

Para validar a correção do algoritmo e analisar o impacto de diferentes estruturas de dados no desempenho do sistema, o projeto foi desenvolvido em três fases. Foram desenvolvidas três variantes do LZW-HX, uma versão de pesquisa linear, com foco nas regras mencionadas no guião, uma versão com Hash Tables para reduzir o espaço de procura, e uma versão baseada em Árvores de Prefixos (Tries) para aumentar a velocidade de processamento. Adicionalmente, também foi implementado o algoritmo LZW clássico [2] para servir como referência para comparação em termos de desempenho e rendimento.

1.2 Arquitetura do Código

A implementação foi estruturada para permitir a existência de múltiplas variantes do algoritmo no mesmo projeto sem duplicação da lógica comum entre elas. Para isso, isolamos as definições gerais, como as estruturas de dados de estatísticas, resultados de procura e o formato dos padrões, num ficheiro que engloba essas mesmas definições (lzw_core.h).

Esta separação permitiu que cada variante fosse tratada como um módulo independente. Cada uma das três variantes do algoritmo possui a sua própria diretoria de ficheiros, contendo:

- Um ficheiro de cabeçalho (ex: lzw_linear.h), que herda as estruturas do ficheiro core e define o tipo de dicionário específico daquela abordagem.
- Um ficheiro de implementação .cpp (ex: lzw_HashTable.cpp), que contém a lógica complexa de procura e atualização do dicionário.
- Um ficheiro que serve como ponto de entrada (main.cpp), responsável exclusivamente pelo parsing de argumentos, leitura dos ficheiros, e a apresentação do painel de estatísticas.

1.3 Gestão de Memória e Processamento por Blocos

Conforme sugerido nas diretrizes do projeto, o processamento direto em ficheiro foi evitado devido à sua ineficiência. Em vez disso, foi implementado um sistema de leitura e processamento em memória baseada em blocos.

Por defeito, o ficheiro de entrada é segmentado e lido para um buffer de 64 Kbyte (`std::vector<uint8_t>`). Cada bloco é codificado em memória pela função `encode_block`. Esta estratégia não só garante a redução do tempo gasto em operações de leitura e escrita no disco, como também prepara a arquitetura futuras extensões, como por exemplo, a paralelização através de multi-threading. Durante o processamento de cada bloco, a ferramenta regista o tamanho do bloco processado, que é 16 bits para índices simples, e 24 bits para índices com tamanho $K > 1$, e simula o tamanho do buffer de saída, permitindo o cálculo da taxa de compressão e do rendimento sem a necessidade de gerar um ficheiro binário comprimido no disco.

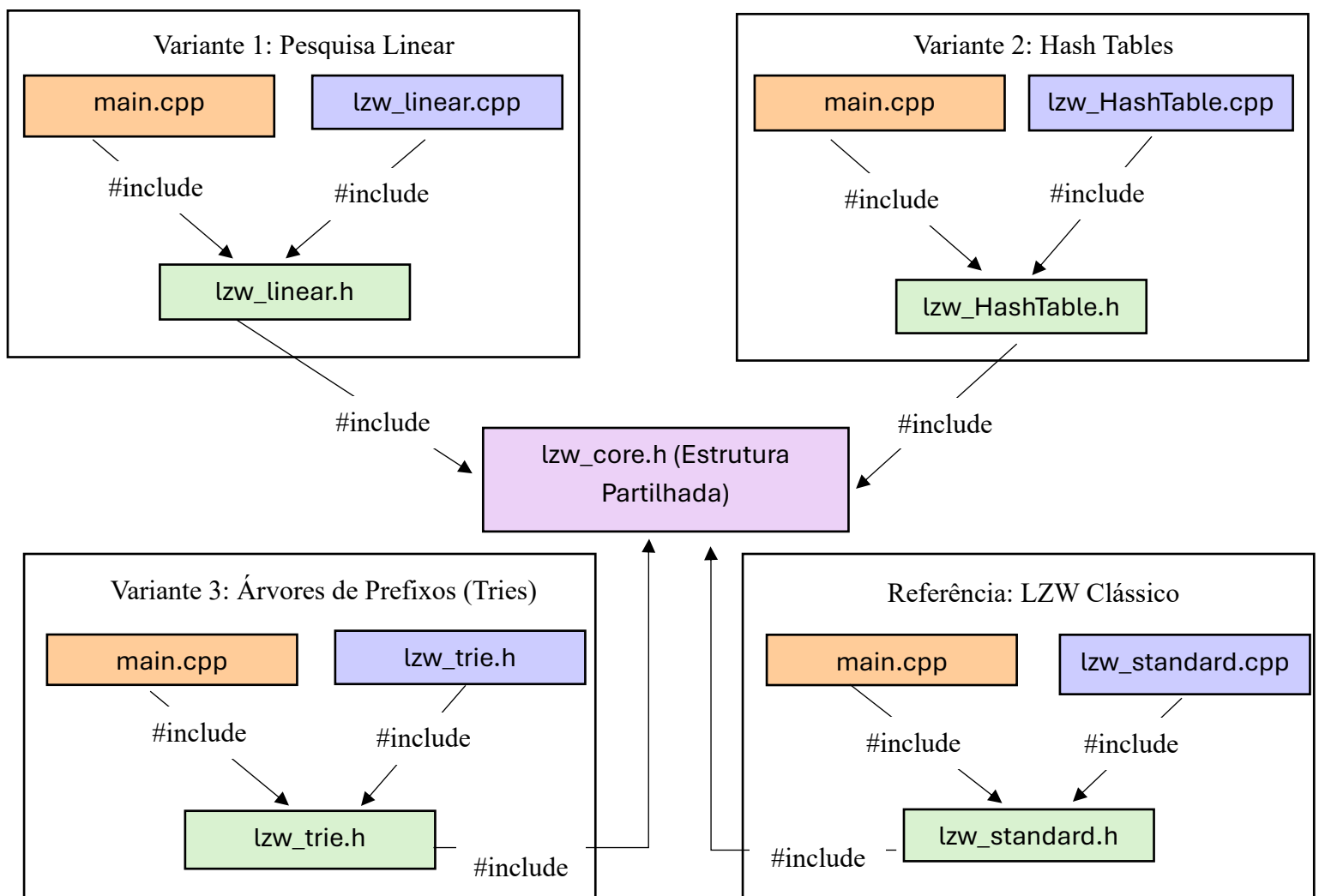


Figura 1 - Diagrama de blocos

2. Estratégias Escolhidas e Mecanismos Adotados

A complexidade lógica do LZW-HX com a política de recurso exigiu mecanismos de procura avançados. As três estratégias implementadas resolvem este desafio com diferentes compromissos arquiteturais.

2.1 Versão Base: Pesquisa Linear

A primeira estratégia implementada utilizou um dicionário suportado por apenas um simples vetor (`std::vector<Pattern>`). Nesta versão, a procura pelo maior padrão (Pa ou Pb) e a procura por candidatos a substituição, implica iterar por todas as entradas do dicionário.

- Garante que as regras do algoritmo LZW-HX são respeitadas. Para além disso, como os padrões são guardados como strings completas, verificar se “AA” é prefixo de “AAB” é trivial.
- A complexidade temporal da procura é $O(N.L)$ onde N é o tamanho do dicionário e L é o comprimento do padrão. À medida que o dicionário cresce em direção ao limite de 65536 entradas, o tempo de execução do bloco aumenta drasticamente, sendo esta versão inadequada para ficheiros de grandes dimensões.

2.2 Versão Otimizada: Otimização com Hash Tables

Para ser possível otimizar o tempo de execução do bloco, introduziu-se uma otimização utilizando Hash Tables. O dicionário foi expandido para incluir um array de 256 vetores de índices (`std::vector<int> buckets [256]`).

- Quando um novo padrão é inserido, o seu índice é guardado no bucket correspondente ao seu primeiro símbolo ($C_i[0]$).
- Ao processar a sequência de entrada, a pesquisa é imediatamente direcionada apenas para o bucket correto, reduzindo o espaço de procura, em média, por um fator de 256. Esta otimização manteve a implementação da política de recurso, que foi implementada na versão base, mas ao mesmo tempo reduziu drasticamente o tempo de execução do algoritmo, tornando-o viável para ficheiros de grandes dimensões.

2.3 Versão Final: Árvores de Prefixos (Tries)

De forma a explorar os limites do desempenho, desenvolveu-se uma terceira versão que visa a implementação de árvores de prefixos. Estruturalmente, a Trie permite

encontrar padrões num tempo de $O(L)$, independentemente do tamanho do dicionário (N), bastando descer pelos nós correspondentes aos bytes lidos.

No entanto a sua adaptação ao algoritmo LZW-HX revelou ser mais complexa:

- Numa Trie convencional, apenas o nó final, a folha, possui um índice de dicionário válido. Contudo, o guião exige que prefixos de padrões maiores possam ser emitidos para a saída.
- Para contornar esta limitação, recorreu-se á implementação de uma estrutura de dados TrieNode com um `descendent_index`. Durante cada inserção, cada padrão deixa um “rastros” do seu índice em todos os nós intermédios por onde passa. Assim, se a pesquisa parar num nó que não seja o final de um padrão, o algoritmo utiliza o índice guardado no “rastros” (`descendent_index`) desse nó para identificar um padrão válido que o contenha. Isto permite cumprir a regra do LZW-HX que exige a utilização de sub-padrões na saída. Para manter a integridade da árvore, implementamos um sistema que recalcula de baixo para cima, ou seja, sempre que um padrão é removido, o algoritmo sobe a árvore e atualiza o `descendent_index` nos nós superiores. Isto garante que, se houver outros padrões que passem por esses mesmos nós, a continuidade da pesquisa não é afetada pela eliminação de uma entrada.
- Apesar do tempo de execução ser ainda menor quando comparado com as duas versões anteriores, a Trie, é por natureza, ineficiente a realizar pesquisas baseadas no menor comprimento para a política de recurso. Para garantir o correto funcionamento desta regra, a implementação final adota uma estratégia, em que se utiliza a eficiência do Trie para a pesquisa de padrões, mas recorremos a um varrimento linear do vetor de entradas apenas quando o dicionário atinge a capacidade máxima.

3. Explicação das funções implementadas

Esta secção analisa as funções que implementam as regras específicas do LZW-HX. Detalhamos como a estrutura de dados gere a atualização do dicionário e de que forma o algoritmo resolve ambiguidades quando existem múltiplos padrões candidatos para a mesma sequência de entrada.

3.1 update_dictionary: A Política de Substituição e de Recurso

A função `update_dictionary` gere o ciclo de vida dos padrões no dicionário. Diferente do LZW tradicional, onde muitas vezes a tabela reinicia quando atinge o limite `T`, o LZW-HX, mantém uma aprendizagem contínua. O fluxo lógico começa na construção do candidato `Ci` (junção de `Pa` e `Pb`) e divide-se em três cenários de decisão.

- **Casos 1 e 2 (Substituição do Prefixo)**

→ O algoritmo tenta primeiro substituir um padrão mais antigo (`Pold`) do qual `Ci` seja um super-padrão, tendo `Pold` como prefixo. Se não existir e houver espaço, `Ci` é inserido, esta abordagem garante que o dicionário cresce indefinitivamente, formando sequências cada vez maiores.

- **Caso 3 (Dicionário Cheio – Política de Recurso)**

→ Quando o dicionário atinge as `T` entradas, o algoritmo não para de aprender. Implementamos uma política de substituição que procura o padrão que partilhe o prefixo de `Ci`, tenha o menor comprimento e, em caso de empate, o menor número de utilizações. Esta regra garante que os padrões curtos e pouco úteis são “reciclados” para dar lugar a novos candidatos `Ci` potencialmente mais adaptados à entropia local do ficheiro.

Nesta função também são utilizados contadores de utilização (`use_count`) que vão acumulando valores indefinitivamente. Para mitigar isto, a função invoca periodicamente, a cada 4096 substituições, o método `age_dictionary_counters()`, que aplica um bitshift para a direita ($\gg 1$) a todos os contadores. Isto realiza uma limpeza “parcial”, já que reduz os pesos para metade.

3.2 find_longest_match: Ambiguidade e Resolução de Sub-Padrões

O algoritmo LZW-HX também obriga o suporte de sub-padrões, identificados por um tuplo `<índice, comprimento>`. A função não se limita a devolver o primeiro match válido, ela também é capaz de resolver ambiguidades.

Perante a ambiguidade de existirem múltiplos candidatos no dicionário que contêm a sequência atual como prefixo, como por exemplo, “AB” sendo prefixo de “ABBA” e “ABAB”, o LZW-HX resolve o conflito ao priorizar o padrão que demonstrou ter uma maior utilidade estatística (maior `use_count`) até ao momento. Este mecanismo

de desempate garante que o codificador seleciona sempre o padrão que provou ser estatisticamente mais fiável ao longo do processamento do ficheiro, maximizando a probabilidade de prever corretamente os símbolos seguintes.

4. Resultados dos testes efetuados

O comportamento das diferentes implementações foi testado e comparado com o LZW convencional utilizando o ficheiro teste.txt (296 KB; 4.58 bits/símbolo). A análise focou-se em quatro cenários, sendo o primeiro apenas para validar a implementação das regras, e os restantes de carga de dicionário, permitindo medir a escalabilidade e a robustez de cada variante. Para o primeiro cenário, utilizamos um segundo ficheiro teste_2.txt (1 bit/símbolo), que contém a sequência de teste fornecida no enunciado (AAABABBABABAABBABBAB).

4.1 Cenário de Validação Teórica (Alfabeto Reduzido: K=2)

Para validar a correta implementação das regras de sub-padrões e a funcionalidade de inicialização customizada do dicionário exigida no guião, a ferramenta foi testada com a opção --k2. Esta configuração restringe o alfabeto base aos símbolos "A" e "B", pré-carregando o dicionário com os padrões {"A", "B", "AA", "BB", "AB", "BA"}.

Utilizando a sequência de teste fornecida no enunciado (AAABABBABABAABBABBAB), a execução no modo --debug permitiu verificar que o algoritmo processa os candidatos C_i e resolve as ambiguidades de prefixos em total conformidade com a tabela 1 do guião. Esta validação provou que a lógica de de índices de descendentes implementada na Trie, bem como a pesquisa nas Hash Tables, não altera o comportamento do algoritmo LZW-HX, garantindo que o output é igual ao esperado pelo modelo matemático.

```
→ Tries ./lzw_hx teste_2.txt --k2 --debug
Encoding (Trie version)... Please wait.

Output: (2) [Debug: pos=0 | Pa=2 (len:2) | Pb=4]
Output: (4) [Debug: pos=2 | Pa=4 (len:2) | Pb=4]
Output: (4,2) [Debug: pos=4 | Pa=4 (len:2) | Pb=5]
Output: (5) [Debug: pos=6 | Pa=5 (len:2) | Pb=5]
Output: (5,2) [Debug: pos=8 | Pa=5 (len:2) | Pb=5]
Output: (5,2) [Debug: pos=10 | Pa=5 (len:2) | Pb=6]
Output: (6) [Debug: pos=12 | Pa=6 (len:4) | Pb=3]
Output: (3) [Debug: pos=16 | Pa=3 (len:2) | Pb=6]
Output: (6,2) [Debug: pos=18 | Pa=6 (len:2) | Pb=-1]
```

Figura 2 - Validação da extração dos prefixos Pa e Pb no cenário de alfabeto reduzido (Modo Debug).

4.2 Cenário Base (Dicionário: 65536 entradas)

Neste cenário avaliou-se a capacidade da estrutura de dados sem atingir o limite máximo do dicionário.

Tabela 1 - Comparação do desempenho das estruturas de pesquisa num cenário de validação teórica (Alfabeto reduzido, K=2).

Versão	Tempo de Execução	Taxa de Compressão	Rendimento	Comprimento Médio do Padrão Inserido
Pesquisa Linear	60.14 s	1.75:1	100.44%	10.43 bytes
Hash Table (Buckets)	3.44 s	1.75:1	100.44%	10.43 bytes
Trie (Prefix Tree)	0.26 s	1.75:1	100.57%	10.43 bytes
Standard LZW	0.04 s	2.19:1	125.81%	5.40 bytes

A evolução da arquitetura cumpriu os objetivos teóricos. A passagem de pesquisa linear para Hash Table reduziu o tempo em cerca de 94%, provando a eficiência da pesquisa. A implementação de árvores de prefixo, reduziu esse mesmo tempo ainda mais, atingindo cerca de 0.26s, o que demonstra a superioridade de procura em tempo $O(L)$. O facto de as três versões apresentarem exatamente o mesmo rácio de compressão de 1.75:1 demonstra que a introdução de *buckets* e a introdução de árvores de prefixo, acelerou o processamento sem comprometer a lógica do LZW-HX, validando a correção da implementação face à versão inicial de pesquisa linear.

Por outro lado, observa-se que o LZW-HX constroi padrões muito mais longos, graças à junção de Pa e Pb. Contudo a taxa de compressão do LZW tradicional é superior, isto porque, enquanto o LZW tradicional tem um índice de 16 bits, o LZW-HX é forçado a utilizar 24 bits (índice + tamanho), sempre que utiliza sub-padrões ou padrões multi-símbolo.

4.3 Cenário com Dicionário Reduzido (4096 entradas)

Para demonstrar o funcionamento da política de recurso, o dicionário foi reduzido para apenas 4096 entradas, resultando em mais de 83000 substituições de padrões curtos.

Tabela 2 - Métricas de desempenho e compressão do algoritmo LZW-HX face ao LZW Clássico (Dicionário: 65536 entradas).

Versão	Tempo de Execução	Substituições	Taxa de Compressão	Rendimento
Pesquisa Linear	14.14 s	83.462	1.14:1	65.63%

Hash Table (Buckets)	0.81 s	83.462	1.14:1	65.63%
Trie (Prefix Tree)	3.52 s	83.084	1.18:1	67.70%

Neste cenário é possível observar que a Hash Table superou a Trie em termos de velocidade. Isto é um excelente indicador das propriedades inerentes de cada estrutura de dados. Como a regra de substituição exige encontrar o padrão de menor comprimento no dicionário, a Hash Table resolve o problema rapidamente ao iterar sobre um bucket pequeno.

A Trie, no entanto, não consegue ordenar nós folha por profundidade de forma eficiente de cima para baixo, sendo obrigada a percorrer o vetor secundário de forma linear sempre que o dicionário enche. Isto demonstra que a Hash table oferece o desempenho mais equilibrado em condições onde a memória é restrita.

4.4 Cenário de Adaptação (4096 entradas com Reset)

Foi também testada a opção reset, que limpa totalmente o dicionário sempre que atinge o limite de 4096 entradas, em vez de utilizar a política de substituição de prefixos do LZW-HX.

Tabela 3 - Impacto da política de reinicialização (reset) no tempo de execução e rendimento do dicionário (Limite: 4096 entradas).

Versão	Tempo de Execução	Veze que encheu	Taxa de Compressão	Rendimento (Yield)
Pesquisa Linear	8.61 s	19	1.08 1	62.39%
Hash Table (Buckets)	0.49 s	19	1.08:1	62.39%
Trie (Prefix Tree)	0.19 s	19	1.09:1	62.53%

Observa-se que, com o dicionário pequeno e a funcionalidade de reset ativa, os tempos de execução baixam drasticamente em comparação com a política de substituição. Isto acontece porque o dicionário passa mais tempo em estados “vazios” ou semi-preenchidos, o que acelera as procurar lineares.

Mais uma vez, a árvore de prefixo é a que obteve o menor tempo de execução. Isto acontece porque o reset elimina a necessidade de percorrer o vetor para achar padrões mais curtos.

O rendimento é o mais baixo de todos os testes anteriormente realizados. Isto demonstra que limpar o dicionário 19 vezes, impede o algoritmo de achar padrões que sejam longos e úteis. O LZW-HX beneficia muito mais da sua política de substituição do que um reset total, confirmando que a lógica de limpeza de padrões do guião é melhor para a manutenção de um dicionário relevante.

4.4 Cenário com ficheiros pré-comprimidos (JPG, ZIP, PDF)

Para além dos testes anteriormente realizados, também foram realizados testes em ficheiros nativamente comprimidos, como JPG, ZIP e PDF. Estes ficheiros caracterizam-se por ter uma entropia muito próxima do limite teórico, isto significa que os dados se comportam quase como ruído aleatório, já que há uma escassez de padrões repetitivos.

Tabela 4 - Resultados da simulação com o ficheiro zip

Versão / Arquitetura	Tempo de Execução	Substituições	Taxa de Compressão	Rendimento (Yield)
Hash Table (Buckets)	0.47 s	16.024	0.72:1	70.89%
Trie	0.22 s	16.024	0.72:1	70.89%
Pesquisa Linear	71.03 s	16.024	0.72:1	70.89%
Standard LZW	0.04 s	N/A	0.81:1	79.81%

Tabela 5 - Resultados da simulação com o ficheiro JPEG

Versão / Arquitetura	Tempo de Execução	Substituições	Taxa de Compressão	Rendimento (Yield)
Hash Table (Buckets)	4.97 s	151.891	0.61:1	60.76%
Trie (Prefix Tree)	116.75 s	151.892	0.61:1	60.84%
Pesquisa Linear	799 s	151.891	0.61:1	60.76%
Standard LZW	0.11 s	N/A	0.80:1	79.45%

Tabela 6 - Resultados da simulação com o ficheiro pdf

Versão / Arquitetura	Tempo de Execução	Substituições	Taxa de Compressão	Rendimento (Yield)
Hash Table (Buckets)	7.12 s	252.113	0.63:1	63.20%
Trie (Prefix Tree)	199.66 s	252.066	0.63:1	63.23%
Pesquisa Linear		-	-	-
Standard LZW	0.10 s	N/A	0.80:1	80.19%

A análise destes ficheiros revelou três conclusões fundamentais sobre o algoritmo e as estruturas de dados:

Como era esperado teoricamente, tentar comprimir um ficheiro já comprimido, resulta na expansão do ficheiro original. Em todos os testes, o rácio de compressão foi inferior a 1. Isto ocorre porque o algoritmo não consegue encontrar padrões longos o suficiente para justificar o custo do envio dos códigos. O LZW clássico obteve melhores

resultados, mas apenas porque emite códigos fixos de 16 bits, ao contrário do algoritmo LZW-HX, que é forçado a emitir tuplos com mais bits, com uma média de 20 bits nestes testes, para representar padrões curtos.

Por outro lado, também se verificou um contraste no desempenho da árvore de prefixos, dependendo do estado do dicionário. No ficheiro ZIP (97 KB), o dicionário não atingiu o limite, e por esse mesmo motivo, a Trie superou a Hash Table. No entanto, nos ficheiros JPG e PDF, o dicionário encheu e forçou, respetivamente, mais de 150000 e 250000 operações de substituição. Sob esta carga, a performance da Trie foi muito baixa, tendo demorado 200 segundos, enquanto comparada com a Hash Table, que apenas demorou 7.1 segundos. Isto prova que a Trie é excecional para pesquisas de prefixos, mas é ineficiente a executar procuras para encontrar os padrões mais curtos e menos usados, o que exige percorrer nós e folhas iterativamente. Por outro lado, a Hash Table, ao limitar a pesquisa a pequenos buckets, consegue isolar o impacto das substituições.

5. Conclusões e Melhorias

Concluindo, a implementação e análise do algoritmo LZW-HX permitiram concluir que esta variante de compressão depende da arquitetura de dados escolhida, visto que a exigência de procurar os prefixos mais longos (P_a e P_b) inviabiliza abordagens básicas em termos computacionais, como ficou demonstrado pelos 8.61 segundos registados na pesquisa linear durante os testes de limite de dicionário.

A transição para estruturas de dados mais complexas de modo a diminuir o tempo de execução revelou que não existe uma estrutura de dados perfeita para todos os cenários. Por um lado, a árvore de prefixos resolveu o problema de pesquisa atingindo tempos de execução muito menores na compressão de texto que qualquer outra arquitetura implementada, graças à procura em tempo $O(L)$. Por outro lado, quando sujeita a testes com ficheiro pré-comprimidos, como o JPG e o PDF, revelou ser ineficiente comparado à Hash Table, porque é forçada a executar milhares de procuras para cumprir a política de substituição LZW-HX. Portanto a arquitetura baseada em Hash Tables demonstrou ser a mais resiliente e mais equilibrada de todo o projeto, garantindo tempos de execução baixos sem sacrificar a compressão.

Por outro lado, no que toca à gestão de memória, os testes provaram que usar o reset prejudica a compressão, baixando o rendimento para 62%. Concluimos deste modo, que a política de substituição do LZW-HX é muito superior, visto que ao substituir os padrões menos úteis, o algoritmo consegue adaptar-se à entropia do ficheiro sem destruir o histórico de padrões longos já apresentados.

O LZW-HX apresenta-se como uma alternativa robusta e eficiente face ao LZW Clássico, embora a regra de atualização exija um processamento lógico mais complexo, mas o uso de Tries reduz a complexidade resultando num algoritmo capaz de manter um dicionário adaptativo, operando dentro de um limite de memória.

Resumindo, como trabalho futuro e para melhoria da implementação, sugere-se a exploração de uma estrutura de dados híbrida. O objetivo seria juntar a velocidade de pesquisa das Tries a um mecanismo de controlo de estado (como uma Doubly Linked List ou uma LRU Cache), reduzindo assim o esforço exigido pelas substituições. Isto permitiria otimizar o uso da memória cache e, consequentemente, tornar a execução muito mais eficiente em termos de consumo de energia.

.6. Referências Bibliográficas

- [1] TP1 SRAM 2025-2026_v08
- [2] SRAM_C_CodificaçãoPadrões_q

7. Declaração de Uso de IA e Responsabilidade

Ferramentas utilizadas: Gemini 3.1 Pro.

Âmbito da Utilização:

A utilização de Inteligência Artificial serviu como apoio ao desenvolvimento, sobretudo na modulação da arquitetura, na implementação de boas práticas de programação e também na construção do algoritmo LZW tradicional.

Detalhes da Interação com IA

Modulação da arquitetura

A arquitetura modular do projeto foi planeada com o apoio de ferramentas de IA, o que permitiu organizar as diferentes variantes do algoritmo de forma independente e seguir as melhores práticas de desenvolvimento em C++.

- Prompt Enviado: "Poderia ajudar-me a estruturar o código? Quero que ele seja mais organizado e limpo. Qual é a estrutura padrão para um projeto em C++ como este, por exemplo?"

- **Resumo da Resposta da IA:** A IA recomendou a transição de um único ficheiro, para uma estrutura dividida por três partes, fundamentada nos seguintes princípios:
 - **Separação de Definição e Implementação:** Utilização de ficheiros de cabeçalho (.h) e ficheiros .cpp para a lógica (Implementação). Esta prática evita dependências circulares e melhora o tempo de compilação.
 - **Função Principal / Coordenador Main:** O ficheiro main.cpp deve atuar apenas como coordenador, ou seja, limita-se a gerir o ciclo de vida da execução.
 - **Header comum:** Criação do lzw_core.h para evitar a duplicação de estruturas de dados comuns (Statistics, SearchResult) entre as diferentes versões dos algoritmos.
- **Justificação da Aplicação:** Com base nesta resposta, o grupo implementou a estrutura sugerida pela IA, isolando as variantes (Linear, Buckets, Tries). Esta organização permitiu realizar os testes de performance de cada versão do algoritmo, de forma independente e limpa, garantindo que, por exemplo, a complexidade da Trie não interfere na simplicidade da versão de pesquisa linear.

Implementação do LZW Clássico

Para validar a eficiência do algoritmo experimental LZW-HX, o grupo decidiu, implementar uma versão do LZW tradicional para comparar com as versões desenvolvidas do algoritmo LZW-HX. A IA foi consultada para ajudar na tradução da lógica teórica do LZW para o ambiente modular de C++ já estabelecido no projeto.

- **Prompt Enviado:** "Uma vez que já tenho as minhas outras versões implementadas (Pesquisa Linear, Tries e Tabela de Hash), pretendo ainda comparar com o LZW tradicional. O objetivo é observar as diferenças entre a implementação do LZW-HX experimental e um algoritmo que já existe. Tendo em conta as outras partes do código que já possuo, poderias ajudar-me a implementar a lógica do LZW padrão?"
- **Resumo da Resposta de IA:** A IA forneceu a estrutura lógica baseada no princípio P + C (Prefix + Character), destacando as diferenças fundamentais que deveriam ser observadas nos testes:

- Diferenciação de Lógica: Enquanto o LZW-HX utiliza a concatenação de dois padrões (Pa|Pb), o LZW Clássico utiliza o padrão atual (P) e o próximo símbolo individual (C).
- Eficiência de Codificação: A IA explicou que o LZW Clássico utiliza índices fixos (tipicamente 16 bits), ao passo que o LZW-HX introduz a complexidade de tuplos de 24 bits (índice + comprimento).
- Crescimento do Dicionário: Foi realçado que o LZW Clássico cresce um símbolo de cada vez, permitindo comparar a velocidade de preenchimento do dicionário face ao crescimento agressivo do LZW-HX.
- Justificação da Aplicação: A implementação sugerida pela IA foi integrada no ficheiro `lzw_standard.cpp`. Esta colaboração foi fundamental para permitir uma análise crítica e rigorosa na Secção 4 do relatório, onde se concluiu que, embora o LZW-HX gere padrões mais longos, a simplicidade do LZW Clássico resulta frequentemente numa melhor taxa de compressão devido ao menor número de bits por código enviado.

Declaração de Responsabilidade:

Os alunos Luís Baptista e Bernardo Salgado declaram assumir total pela autoria do código fonte e pelos conteúdos técnicos apresentados neste relatório. Confirmamos que compreendemos o funcionamento da arquitetura implementada, bem como todas as decisões algorítmicas tomadas para a concretização das variantes do LZW-HX (Linear, Buckets e Trie). Justificamos o uso de Inteligência Artificial como uma ferramenta de auxílio, focada na estruturação de ficheiros, indentação de código e implementação do algoritmo LZW Clássico para fins comparativos, garantindo que toda a lógica experimental do LZW-HX foi validada e ajustada manualmente pelo grupo.